JAVA PROGRAMMING · DAY 2

Variables, Input & Conditions

Data Types, Scanner Input, Real-World Example, if-else-elseif, Switch Case

5 Programs · Explained Step-by-Step for Easy Understanding

1 Variables & Data Types

A **variable** is a named box that stores a value. Java is a "typed" language — every variable must declare what **type** of value it will hold: **int** for whole numbers, **String** for text, and **boolean** for true/false.

Program 1 — Declare Variables and Print Them

```java
public class Main {
    public static void main(String[] args) {
        int age = 10;
        String name = "Saul Goodman";
        boolean isStudent = true;

        System.out.println("my age is " + age);
        System.out.println("my name is " + name);
        System.out.println("whether I am Student ? " + isStudent);
    }
}
```

CONSOLE OUTPUT

```
my age is 10
my name is Saul Goodman
whether I am Student ? true
```

What's happening here?

- **int age = 10;** → stores a whole number (no decimals).
- **String name = "Saul Goodman";** → stores text, always inside double quotes.
- **boolean isStudent = true;** → stores only **true** or **false**.
- The **+** symbol joins ("concatenates") text with a variable's value when used inside **println**.

2 Taking Input from the User (Scanner)

So far our values were fixed in the code. To let the **user** type values while the program runs, Java gives us the **Scanner** class from the **java.util** package.

Program 1 — Read Age, Name and Student Status from User

```java
import java.util.*;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.println("Enter the age : ");
        int age = sc.nextInt();
        sc.nextLine();

        System.out.println("Enter the name : ");
        String name = sc.nextLine();

        System.out.println("Enter whether you are a student ? : ");
        boolean isStudent = sc.nextBoolean();

        System.out.println("age : " + age);
        System.out.println("name : " + name);
        System.out.println("Student status : " + isStudent);
    }
}
```

CONSOLE OUTPUT (SAMPLE RUN)

```
Enter the age :
21
Enter the name :
Saul Goodman
Enter whether you are a student ? :
true
age : 21
name : Saul Goodman
Student status : true
```

What's happening here?

- **import java.util.*;** → brings in the Scanner class so we can use it.
- **Scanner sc = new Scanner(System.in);** → creates a "reader" that listens to keyboard input.
- **sc.nextInt()** reads a number, **sc.nextLine()** reads a full line of text, **sc.nextBoolean()** reads **true**/**false**.
- **Why the extra sc.nextLine() after nextInt()?** When you type a number and press Enter, the Enter key (newline) is left behind unread. The extra **sc.nextLine()** clears it away so the next **nextLine()** doesn't accidentally read an empty line instead of the name.

3 Real-World Example — Employee Details

Let's use what we learned about variables to model something practical: storing and printing an employee's record.

Program 1 — Store and Print Employee Details

```java
public class Main {
    public static void main(String[] args) {
        int empId = 100;
        String empName = "JohnCena";
        String empGender = "Male";
        long empMobile = 9999999999L;
        double empSalary = 17000;
        boolean isMarried = true;

        System.out.println("Employee ID : " + empId);
        System.out.println("Employee Name : " + empName);
        System.out.println("Employee Gender : " + empGender);
        System.out.println("Employee Mobile No : " + empMobile);
        System.out.println("Employee Salary : " + empSalary);
        System.out.println("Employee Married Status : " + isMarried);
    }
}
```

CONSOLE OUTPUT

```
Employee ID : 100
Employee Name : JohnCena
Employee Gender : Male
Employee Mobile No : 9999999999
Employee Salary : 17000.0
Employee Married Status : true
```

What's happening here?

- This program mixes **int**, **String**, **long** (for the big mobile number), **double** (for salary), and **boolean** in one record — just like a real employee database row.
- **long** is used for the mobile number because it has 10 digits, which is too large for a normal **int** to safely hold as a phone number literal (hence the **L** at the end).
- **double** is used for salary since salaries can involve decimals (like 17000.50).

4 Conditional Statements (if – else if – else)

Conditional statements let the program make **decisions**. Java checks each condition from top to bottom and runs the first block whose condition is true.

Program 1 — Grade Calculator Based on Marks

≥90 → A grade | ≥80 → B grade | ≥70 → C grade | Below 70 → Study more!

```java
import java.util.*;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.println("enter the mark");
        int mark = sc.nextInt();

        if (mark >= 90) {
            System.out.println("A grade");
        } else if (mark >= 80) {
            System.out.println("B grade");
        } else if (mark >= 70) {
            System.out.println("C grade");
        } else {
            System.out.println("parama padi daaa!!! ");
        }
    }
}
```

CONSOLE OUTPUT (SAMPLE RUN)

```
enter the mark
65
parama padi daaa!!!
```

Fixed from your original code: System.out.pritln → System.out.println (typo), and added the missing semicolon **;** after the last println statement.

What's happening here?

- Java checks conditions **in order**: if the first **if** is false, it moves to **else if**, and so on. Only one block runs.
- **else** at the end is a catch-all — it runs when none of the above conditions matched.
- Since we check **mark >= 90** first, we don't need to also write **mark < 100** — anything that reaches the second condition is already known to be below 90.

5 Control Statements (switch case)

A **switch** statement is a cleaner alternative to a long else-if ladder when you're comparing one variable against many fixed values.

Program 1 — Print Day Name from Day Number

1 → Monday ... 7 → Sunday

```
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter the day number (1-7): ");
        int day = sc.nextInt();

        switch (day) {
            case 1:
                System.out.println("Monday");
                break;
            case 2:
                System.out.println("Tuesday");
                break;
            case 3:
                System.out.println("Wednesday");
                break;
            case 4:
                System.out.println("Thursday");
                break;
            case 5:
                System.out.println("Friday");
                break;
            case 6:
                System.out.println("Saturday");
                break;
            case 7:
                System.out.println("Sunday");
                break;
            default:
                System.out.println("Invalid day number");
        }
    }
}
```

CONSOLE OUTPUT (SAMPLE RUN)

```
Enter the day number (1-7): 3
Wednesday
```

What's happening here?

- **switch(day)** compares the value of **day** against each **case** label.
- **break;** stops the switch once a match is found — without it, Java would keep running the next cases too ("fall-through").
- **default** runs only if none of the cases matched, similar to the final **else** in an if-else ladder.